

Arcaidia — Frontend Specification

For: Lovable (or any frontend generator) **Version:** 1.0 · 2026-09-06 **Target repo:** `apps/web` inside the existing pnpm monorepo

0. Read this first

Arcaidia's backend is complete and tested — contracts, solver, settlement worker, subgraphs. This document specifies **only the frontend**. Do not invent business logic, signing, or settlement behaviour; every number and status shown comes from data structures defined in §7.

Three rules override any design instinct:

1. **Never show a single "complete" state.** A transfer has *two independent* settlement facts — fast and canonical. They must always render as two separate tracks. A progress bar that reaches 100% when the user is paid is wrong and will be rejected.
 2. **Never build two directions.** One component set handles Ethereum→Arc and Arc→Ethereum. The direction is a value, not a route.
 3. **The frontend never signs, prices, or decides anything.** It displays what the backend returns and submits what the user authorises through Privy.
-

1. What Arcaidia is (use this for copy)

Arcaidia is a speed layer over Circle's CCTP. Your money goes into Circle's pipe first; an autonomous agent verifies that independently, prices the wait, and advances you the destination funds from a liquidity vault in seconds. CCTP repays the vault minutes later.

Supporting facts that may be used in copy:

- Crosschain USDC transfer between **Ethereum** and **Arc**, in both directions.
- The user is paid in **seconds**. Canonical settlement completes in **minutes**, afterwards.
- Liquidity providers earn a fee for the minutes their capital is advanced.
- If no solver participates, the transfer still completes at CCTP's own speed. Arcaidia accelerates; it is never required.

Tone: precise, confident, unhyped. This is financial infrastructure. Avoid "revolutionary", "seamless", "next-gen". Prefer concrete numbers over adjectives.

2. Visual direction

Concept: blockchains growing through cyberspace, with electricity arcing between them. Dark, deep, dimensional. The chains are structures; the electricity is value in motion.

Palette

TOKEN	HEX	USE
<code>--void</code>	<code>#05070C</code>	Page ground
<code>--surface</code>	<code>#0B111C</code>	Cards, panels
<code>--surface-raised</code>	<code>#111A29</code>	Elevated panels, modals
<code>--border</code>	<code>#1C2740</code>	Hairlines, panel edges
<code>--text</code>	<code>#E8EEF9</code>	Primary text
<code>--text-dim</code>	<code>#7E8CA8</code>	Secondary text, labels
<code>--electric</code>	<code>#2E9BFF</code>	Neon blue — the fast path, primary action, active state
<code>--electric-glow</code>	<code>#7CC4FF</code>	Blue highlight, glow cores
<code>--gold</code>	<code>#FFB627</code>	Neon gold — canonical settlement, LP yield, value at rest
<code>--gold-glow</code>	<code>#FFD37A</code>	Gold highlight
<code>--danger</code>	<code>#FF4D5E</code>	Rejections, errors, breaches
<code>--success</code>	<code>#3DD68C</code>	Confirmations

Colour carries meaning, not decoration. Blue = the fast path (the advance). Gold = the canonical path (the settlement, the yield). This mapping is used consistently across every page, chart, and the 3D background. Never use blue for a settled state or gold for a pending fill.

Typography

- **Display / headings:** a technical grotesque with character — *Space Grotesk*, *Chakra Petch*, or *Rajdhani*. Tight tracking, weights 600–700.
- **Body:** *Inter* or *Satoshi*, 400/500.
- **Numerals and data:** *JetBrains Mono* or *IBM Plex Mono*, with `font-variant-numeric: tabular-nums` everywhere digits align in columns.

Never render an amount, address, hash, or basis-point figure in a proportional face.

Surface treatment

- Panels: `--surface`, 1px `--border`, radius 12px, subtle inner top highlight (`inset 0 1px 0 rgba(255,255,255,.04)`).
- Glow is earned, not ambient: only active/primary elements emit it. `box-shadow: 0 0 24px -6px var(--electric)` on the primary CTA and the active timeline node only.

- No frosted glass everywhere. One or two elevated surfaces maximum per viewport.
-

3. The 3D background

This is the signature element. It sits behind every page, fixed, non-interactive except for mouse response.

Concept

Two crystalline chain structures — one on each side of the viewport — built from linked nodes that extend slowly into depth, as though growing. Between them, arcs of electricity periodically travel across the gap. Blue arcs travel fast, gold arcs travel slowly.

The two chains are Ethereum and Arc. The arcs are transfers. ****The user is looking at the protocol.****

Implementation

- `@react-three/fiber` + `@react-three/drei`, or hand-written WebGL. Not a video, not a GIF.
- **Mouse tracking:** camera parallax on pointer position, damped (lerp factor ~ 0.05 — it should feel like weight, not like it's stuck to the cursor). Maximum rotation $\pm 6^\circ$, maximum translation ~ 0.4 units. Subtle. The page must never feel like it is sliding around.
- Chain nodes: instanced meshes, emissive material, slight bloom.
- Arcs: animated line geometry or a shader-driven curve, with a bright travelling head and a fading tail. Blue arcs cross in ~ 0.8 s, gold arcs in ~ 4 s — the actual latency ratio of the product.
- Depth fog toward `--void` so structures fade rather than clip.

Hard performance guardrails

The background must never compromise the app.

- Cap `dpr` at `[1, 1.75]`.
- Pause the render loop when `document.hidden`.
- Target 60fps on an M1 MacBook Air; degrade node count if frame time exceeds 20ms.
- Under `prefers-reduced-motion: reduce`: render **one static frame**, no arcs, no parallax.
- Provide a static gradient fallback if WebGL is unavailable.
- Total bundle contribution under 300KB gzipped.

Where it appears

Full-bleed on `/` and `/about`. Dimmed to $\sim 25\%$ opacity behind the app routes (`/transfer`, `/solver`, `/liquidity`) so data stays legible — the app is the point; the background is context.

4. Routes

ROUTE	PURPOSE
/	Landing. Hero with the 3D scene, the one-line pitch, primary CTA to <code>/transfer</code> . Keep it short — one screen of hero, one of "how it works" summary linking to <code>/about</code> , one footer.
<code>/transfer</code>	The product. Create a transfer and watch it settle.
<code>/solver</code>	Live agent activity — the decisions and the inputs behind them.
<code>/liquidity</code>	LP deposit, withdraw, and vault performance.
<code>/about</code>	Paginated walkthrough of how value moves. See §6.
<code>/docs</code>	Static documentation. Architecture, trust assumptions, contract addresses, FAQ.

Persistent chrome: a top bar with the wordmark, the four app routes, and a wallet control (Privy — see §8). No sidebar; this is a four-page app.

5. Page specifications

5.1 `/transfer`

Two-column on desktop ($\geq 1024\text{px}$), stacked on mobile. Left: the form. Right: the timeline.

Transfer form (left)

A single card, Uniswap-like in structure and density.

- **Direction control.** Two chain tiles (Ethereum, Arc) with a circular swap button between them. Clicking swap animates the tiles exchanging position (~250ms). ****This is the only direction control**** — there is no separate reverse flow.
- **Amount field.** Large mono numerals. USDC only. A `MAX` button. Below: the user's balance on the source chain.
- **Recipient field.** Defaults to the connected wallet, editable. Address validation with visible feedback. Show an ENS-style truncation (`0x1234...9f2a`) once valid.
- **Max fee.** A slider or segmented control in basis points (10 / 30 / 50 / 100 bps), with the currency equivalent shown live: *"up to 3.00 USDC"*. Label it plainly: **"Most you'll pay"**.
- **Deadline.** A simple select (30m / 1h / 6h). Collapsed under an "Advanced" disclosure.
- **Quote preview.** Once an amount is entered, show: *You send, Recipient gets, Fee, Estimated arrival*. The fee is returned by the backend, never computed in the UI.
- **Primary button.** States: `Connect wallet` → `Approve USDC` → `Confirm transfer` → `Submitting...`. Full width, blue, with the only glow on the page.

Settlement timeline (right)

The most important component in the application. Two parallel vertical tracks, clearly separated, never merged.

FAST PATH (blue)

- Intent committed
- Verified by solver
- Fee quoted – 1.00 USDC
- Paid – 999.00 USDC
- ✓ You're done

CANONICAL PATH (gold)

- Committed to CCTP
- Awaiting attestation
- Settling
- Liquidity vault repaid
- ⌚ ~12 minutes remaining

Requirements:

- **Two visually distinct columns or lanes.** Blue track on the left, gold on the right, with a visible gutter. They must not read as one sequence.
- The fast track reaching its end shows **"You're done"** with the recipient's amount.
- The canonical track continues afterwards with its own progress and an ETA.
- **Never** render an overall percentage, a single combined progress bar, or a "Complete" badge that depends on only one track.
- Each node: label, timestamp when reached, and a transaction link where one exists.
- Active node pulses gently (blue or gold, matching its track).
- On the fallback path (no solver participated), the fast track shows a neutral **"No solver — settling directly"** state and the gold track pays the recipient. This is a normal outcome, not an error. Do not style it as a failure.

Agent decision panel (below the timeline)

Expandable. When the solver has quoted, show the exact inputs it used:

- Available liquidity · Utilisation (%) · Outstanding exposure
- Settlement backlog · Transport health (badge: Healthy / Slowing / Unavailable)
- Confirmations (4 / 3 format) · Quoted fee vs. the user's ceiling
- Verdict badge: **ACCEPT** (blue) / **REJECT** (red) / **PAUSE** (amber)

Present it as a readout, not a chart — a monospace key/value grid with generous line height. This panel is what makes the agent legible; it should feel like looking at an instrument.

States to design

Empty (no wallet) · Wallet connected, no amount · Quote loading · Quote rejected (show the reason in plain language) · Approving · Submitting · Fast-filled, canonical pending · Fully settled · Fallback path · Insufficient balance · Amount over cap · Deadline expired.

5.2 /solver

A live console. This page is the evidence that an autonomous agent is genuinely operating.

- **Header stats row:** decisions in the last hour · accept rate · median quoted fee · transport health badge.

- **Decision feed.** Reverse-chronological rows, each: timestamp · direction (chain pills with an arrow) · amount · verdict badge · quoted fee bps · reason (for non-accepts). New rows slide in from the top with a brief blue flash.
- **Row expansion.** Clicking a row reveals the full `DecisionInputs` readout (same visual language as the decision panel in §5.1) plus the policy version.
- **Filter chips:** All / Accepted / Rejected / Paused.
- **Small sparkline** of quoted fee over time, blue line on dark, no axes chrome — just the shape.

Design for both a busy feed and an empty one. Empty state: *"No decisions yet — the solver is watching."* with a slow pulse, not a spinner.

5.3 /liquidity

- **Vault selector.** Two tabs: Ethereum vault, Arc vault.
- **Headline metrics** (gold-accented, since this is yield): Total assets · Your position · Share price · Utilisation · APR (if available).
- **Composition bar.** A single horizontal stacked bar showing *Liquid / Advanced (earning) / Protocol fees*. This is the clearest possible expression of what the vault is doing.
- **Deposit / Withdraw panel.** Tabbed, same card density as the transfer form. Show shares received or burned. Withdraw must show **"Available now"** separately from **"Your position"** — they differ when capital is advanced, and that difference is a feature, not an error.
- **Recent activity table:** deposits, withdrawals, fills serviced, reimbursements.

Copy note: explain in one sentence, near the top, why available may be less than position — *"Some of your capital is advanced to a recipient and returns when CCTP settles."*

5.4 /docs

Static, readable, two-column with a sticky in-page nav. Sections: Overview · How it works · The trust assumption · Contract addresses · Fees · FAQ · Links (GitHub, subgraphs).

Keep the 3D background at very low opacity here. Prioritise reading comfort: max 70ch measure.

Must include, verbatim in substance: Arcadia uses an authorised-solver model. The destination vault trusts EIP-712 signatures from allowlisted agent addresses after the agent has independently verified the source chain. This is a disclosed trust assumption, not a claim of trustless crosschain verification.

6. /about — the paginated walkthrough

The centrepiece of the marketing site. A ten-stage, one-stage-at-a-time walkthrough of a transfer, advanced by clicking, arrow keys, or swiping on touch.

Behaviour

- One stage fills the viewport. A progress rail (10 segments) sits at the bottom or side.

- Navigation: click/tap forward, arrow keys, swipe, and clicking any rail segment to jump.
- Each stage animates the 3D background: the camera moves, and the relevant part of the structure illuminates. Stage transitions are ~600ms, eased.
- Deep-linkable: `/about#stage-4`.
- Reduced motion: instant transitions, no camera movement.

The ten stages (copy provided — use closely)

1. **You express an intent.** *"1,000 USDC on Ethereum. Send it to this address on Arc. I'll pay at most 0.3%."*
You sign once. Nothing has moved yet.
2. **Your money enters Circle's pipe.** Arcaidia's router takes your USDC and hands it to CCTP in the same transaction. If the handoff fails, the whole thing reverts and you keep your money. *There is no cancel button after this — and that's the point.*
3. **The commitment is announced.** The router emits an event that can only exist if the funds were committed. Anyone watching can verify it.
4. **An indexer picks it up.** The Graph records the intent within seconds and shows it to the solver. **But the indexer is not trusted** — it says *where to look*, not *what is true*.
5. **The solver checks the chain itself.** It re-reads your transaction directly from an Ethereum node: right router, right amount, right recipient, CCTP actually started, enough confirmations. Only now will it consider risking capital.
6. **It prices the wait.** How much liquidity is free? How much is already advanced? Is Circle settling on time? A deterministic engine — no AI in the decision — returns a fee and a verdict.
7. **It signs a narrow permission.** Not "send money" — a signed statement about *this* intent, to *this* recipient, for *this* amount, valid for 45 seconds.
8. **You're paid.** The destination vault checks the signature and ten other conditions, then sends the recipient 999 USDC. **Seconds have passed. For you, it's over.**
9. **Circle takes its time.** The canonical transfer is still travelling. Minutes, not seconds. The liquidity provider is carrying your transfer on their books.
10. **The vault is repaid.** 1,000 USDC arrives and reimburses the provider, who keeps the 1 USDC fee for the minutes of risk. **And if no solver had shown up? Circle would simply have paid you directly. Arcaidia makes it faster; it is never required.**

7. Data contracts

These types are **already defined in the backend**. Match them exactly — the generated components will be wired directly to these.

```

type Address = `0x${string}`;
type Hex = `0x${string}`;

// Two settlement facts. Never merge them.
type FastStatus = 'PENDING' | 'FAST_FILLED';
type CanonicalStatus = 'PENDING' | 'SETTLED';
type CanonicalOutcome = 'LP_REIMBURSED' | 'RECIPIENT_FALLBACK';

interface Intent {
  intentId: Hex;
  sender: Address;
  recipient: Address;
  inputToken: Address;
  amount: bigint;           // 6 decimals (USDC)
  sourceChainId: number;    // 11155111 Ethereum Sepolia | 5042002 Arc testnet
  destinationChainId: number;
  maxFeeBps: number;
  deadline: number;         // unix seconds
  nonce: bigint;
  sourceTxHash: Hex;
  createdAt: number;
}

interface IntentSettlementState {
  intentId: Hex;
  fastStatus: FastStatus;
  canonicalStatus: CanonicalStatus;
  canonicalOutcome?: CanonicalOutcome;
  fastFilledAt?: number;
  settledAt?: number;
}

type Verdict = 'ACCEPT' | 'REJECT' | 'PAUSE';

interface AgentDecision {
  intentId: Hex;
  verdict: Verdict;
  reason: string;           // e.g. 'ACCEPTED', 'FEE_CEILING_EXCEEDED', 'SETTLEMENT_BACKLOG'
  feeBps: number;
  feeAmount: bigint;
  outputAmount: bigint;
  policyVersion: string;
  decidedAt: number;
  inputsUsed: DecisionInputs;
}

```



```

interface DecisionInputs {
  requestedAmount: bigint;
  availableLiquidity: bigint;
  reserveFloor: bigint;
  outstandingExposure: bigint;
  utilisationBps: number;
  userMaxFeeBps: number;
  sourceConfirmations: number;
  requiredConfirmations: number;
  observationAgeSeconds: number;
  settlementHealth: {
    transport: 'HEALTHY' | 'DEGRADED' | 'UNAVAILABLE';
    oldestUnsettledAgeSeconds: number | null;
    pendingValue: bigint;
    averageSettlementLatencySeconds: number | null;
  };
}

interface VaultState {
  chainId: number;
  vault: Address;
  asset: Address;
  totalBalance: bigint; // everything held, including protocol fees
  totalShares: bigint;
  reserveFloor: bigint;
  outstandingExposure: bigint; // advanced, awaiting reimbursement
  accruedProtocolFees: bigint; // held but owed to treasury – NOT LP capital
  paused: boolean;
  observedAt: number;
}

```

Formatting rules

- All amounts are `bigint` with **6 decimals**. Never use `Number` for an amount — format with a helper that divides by `1_000_000n` and renders with thousands separators and exactly 2 decimals.
- Basis points: display as a percentage to 2dp (`30 bps` → `0.30%`) with the bps value available on hover.
- Addresses: truncate `0x1234...9f2a` , click to copy, with a toast confirmation.
- Timestamps: relative for recent (`4s ago`), absolute on hover.
- **LP assets** = `totalBalance + outstandingExposure - accruedProtocolFees` . Protocol fees are not LP capital and must never be shown as part of an LP's position.

8. Privy integration

Privy is the **authentication and wallet layer only**. It is one component in the app, not the app.

- Package: `@privy-io/react-auth`.
- Wrap the app in `PrivyProvider` with `NEXT_PUBLIC_PRIVY_APP_ID`.
- **Embedded wallets enabled** — users sign in with email and get a wallet. No seed phrase, no MetaMask requirement. This is the demo path.
- Both chains must appear in `supportedChains`, declared via viem's `defineChain`. Arc is not in `viem/chains`:

```
export const arcTestnet = defineChain({
  id: 5042002,
  name: 'Arc Testnet',
  nativeCurrency: { name: 'USD Coin', symbol: 'USDC', decimals: 18 },
  rpcUrls: { default: { http: ['https://rpc.testnet.arc.io'] } },
  blockExplorers: { default: { name: 'Arcscan', url: 'https://testnet.arcscan.app' } },
});
```

****A chain missing from `supportedChains` throws on send — both must be present or one direction of the product breaks.****

- Wallet control in the top bar: `Connect` → once connected, show truncated address, chain badge, balance, and a menu with Copy address / Fund / Disconnect.
- Handle: not connected · connecting · connected on wrong chain (offer to switch) · connected.

Out of scope for the generated frontend: transaction construction, EIP-712 signing of fill authorizations, or any settlement logic. Those exist in the backend and will be wired in.

9. Technical constraints

Non-negotiable, so the output drops into the existing monorepo:

- **Next.js 15 App Router**, React 19, **TypeScript strict**.
- **Tailwind CSS** with the palette in §2 as CSS custom properties in `globals.css`, referenced through Tailwind theme extension. No hard-coded hex values in components.
- `viem` for all chain types. Do **not** add ethers, wagmi-v1, or web3.js.
- Components in `app/` and `components/`, no `pages/` directory.
- No global state library unless genuinely needed — React context is sufficient.
- Every component that renders backend data takes it as **props with the §7 types**. Mock data goes in one `lib/mock-data.ts` file so it can be deleted in a single commit.

Accessibility

- Keyboard navigable throughout; visible focus rings (blue, 2px, 3px offset).
- Contrast: body text $\geq 4.5:1$ against `--void`. Neon on dark is easy to get wrong — check it.
- Respect `prefers-reduced-motion` for the 3D scene, the walkthrough, and all transitions.
- Every icon-only control has an `aria-label`.

10. Definition of done

- ☐ All six routes render with realistic mock data at 1440px, 1024px, and 390px.
- ☐ The 3D background runs at 60fps and degrades correctly under reduced motion and no-WebGL.
- ☐ The settlement timeline shows two independent tracks with **no combined progress indicator**.
- ☐ The direction toggle swaps in place — there is no second route or duplicated form.
- ☐ The decision panel renders every field of `DecisionInputs`.
- ☐ The `/about` walkthrough advances by click, keyboard, and swipe, and is deep-linkable.
- ☐ Amounts are formatted from `bigint` with 6 decimals, in a monospace face, throughout.
- ☐ Privy connect/disconnect works with an embedded wallet, with both chains configured.
- ☐ No business logic in the frontend: no fee calculation, no verdict derivation, no signing.